

CHAPTER – 07

SET

1. What is a Set?

- A **Set** is a **collection of unique elements** stored in a **single variable**.
- **Elements in a set must be immutable and unique** (no duplicates allowed).
- Sets are **unordered and unindexed**, meaning:
 - You **cannot access elements by index**.
 - The order of elements may vary every time you print or iterate.
- Sets are defined using **curly braces {}** or the `set()` constructor.

2. Key Characteristics of Sets

Property	Description
Unordered	No guaranteed order of elements
Unique Elements	Duplicate values are automatically removed
Mutable collection	You can add or remove elements but cannot modify existing ones directly
No Indexing	Cannot access items by index

3. Creating Sets

- Using curly braces:
CODE: `items = {"a", "b", "c", 2}`
- Using `set()` constructor:
`items = set([1, 2, 3])`
- **Note:** To create an empty set, use `set()` not `{}` (which creates an empty dictionary).

4. Important Notes on Boolean and Numeric Values in Sets

- Boolean values `True` and `False` are treated as integers 1 and 0 respectively.

- So in sets, True and 1 are considered the **same element**; similarly for False and 0.
- Example:

```
s = {True, 1, 0, False}
```

```
print(s) # Output: {0, 1}
```

5. Checking Membership – in Keyword

- You can check if an element exists in a set using in or not in.

```
items = {1, 2, 3, "sa", "va", 10.0}
```

```
print("sa" in items) # True
```

```
print("ta" not in items) # True
```

6. Common Set Methods

Method	Description	Example
add(elem)	Adds a single element to the set	items.add(3)
update(iterable)	Adds multiple elements from another set/list/tuple	items.update({4,5})
clear()	Removes all elements from the set	items.clear()
copy()	Returns a shallow copy of the set	new_set = items.copy()
difference(other_set)	Returns elements present in current set but not in other set	a.difference(b) or a - b
remove(elem)	Removes specified element; raises error if not found	items.remove(3)
discard(elem)	Removes specified element; no error if not found	items.discard(3)
union(*others)	Returns a set with all elements from all sets combined	a.union(b) or ` a
pop()	Removes and returns a random element from the set	items.pop()

7. Example Usage of Set Methods

```
a = {1, 2, 3}
b = {2, 4, 5}
c = [6, 7, 8]
d = (9, 10, 11)

a.update(b)
a.update(c, d)

print(a) # {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11}

print(a.difference(b)) # {1, 3, 6, 7, 8, 9, 10, 11}

a.remove(1)

a.discard(100) # No error, even if 100 is not in the set

print(a.union(b)) # Combined sets
```

Practice Set – Sets in Python

1. Create a set with 5 different elements and print its type.
2. Create a set containing duplicate elements like {1, 2, 2, 3, 4, 4, 5} and print the set. What do you observe?
3. Write a program to check if the element "banana" exists in the set {"apple", "banana", "cherry"}.
4. Add an element "orange" to the set {"apple", "banana", "cherry"} and print the updated set.
5. Update the set {"apple", "banana"} by adding elements from a list ["kiwi", "grapes"] and a tuple ("mango", "melon").
6. Write a program to find the difference between two sets:
a = {1, 2, 3, 4, 5} and b = {4, 5, 6, 7, 8}.
7. Demonstrate the difference between remove() and discard() methods by trying to remove an element not present in the set.
8. Write a program to clear all elements from a set.
9. Use the union() method or | operator to combine three sets:
a = {1, 2}, b = {3, 4}, c = {5, 6}.

10. Write a program to remove and print a random element from a set using `pop()`.